API INTEGRATION LAB

AI Tooling, APIs & the GenAI Development Environment

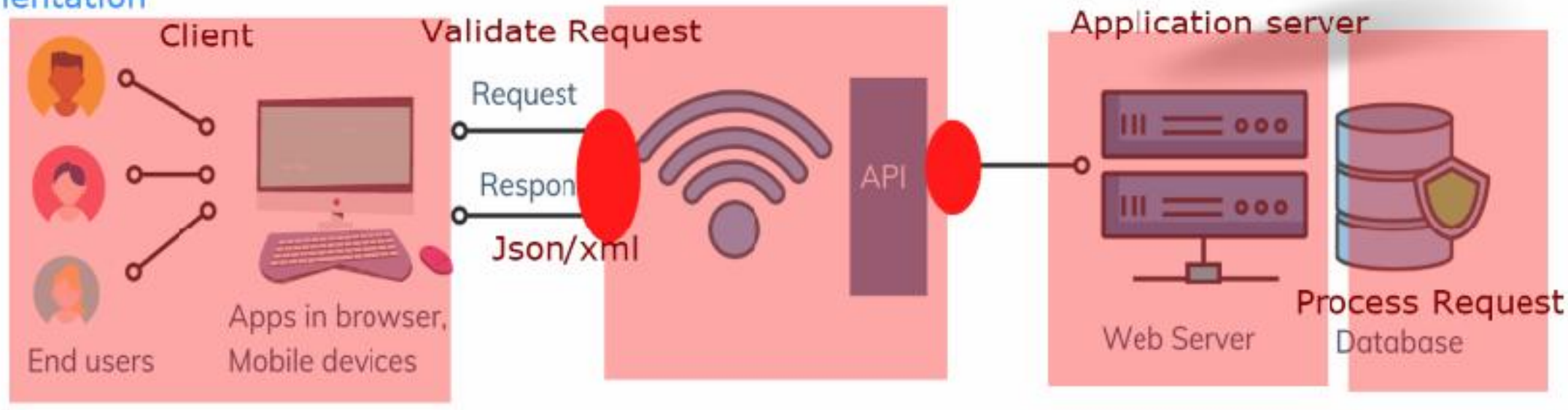
Hands-on with Hugging Face, Google Colab, and the OpenAI, Gemini & Groq APIs — from your first key to a working integration.

Hugging Face & Colab | OpenAI / Gemini / Groq | SDKs & Streaming | Model Parameters | Cost & Tokens

What is an API?

API Endpoints
REquest
Http Methods: GET,POST,PUt,DELETE
Header
RequestBody
Response
Status code
AUthentication: APIKey, JWT token
Documentation

workflow
---
1. Client sends request
2. API validates the request
3. API forwards it to the server/database
4.Server processes the request
5. Server return the result
6.API formats the response (JSON/XML)
7. Client receives the response



Customers

Waiter

Chef



places order

serves the order



forwards order

prepares and hands over the order



User interacting via web

Application Programming Interface

Application server



Client

sends request

gets response

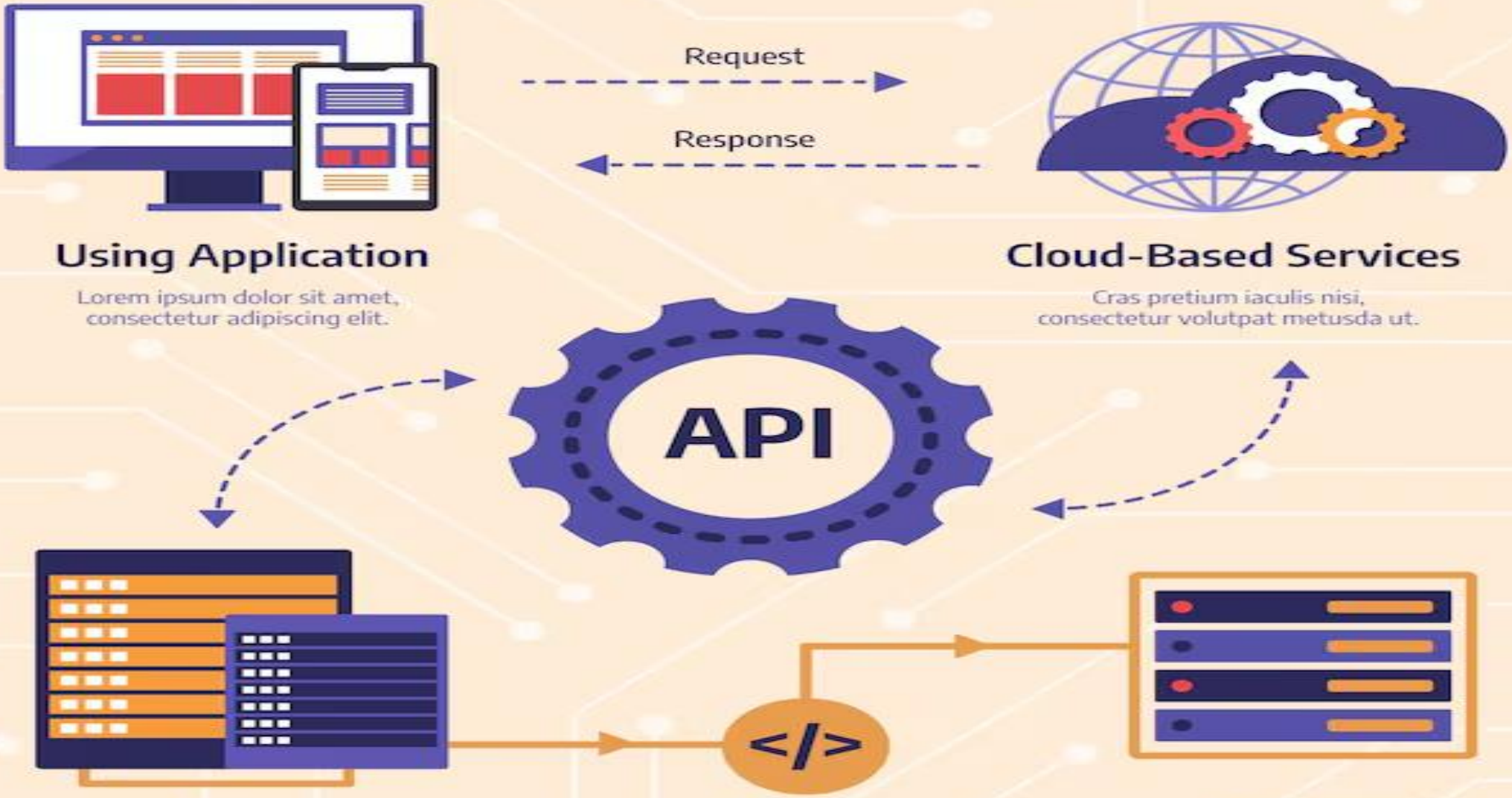


forwards request

processes and responds



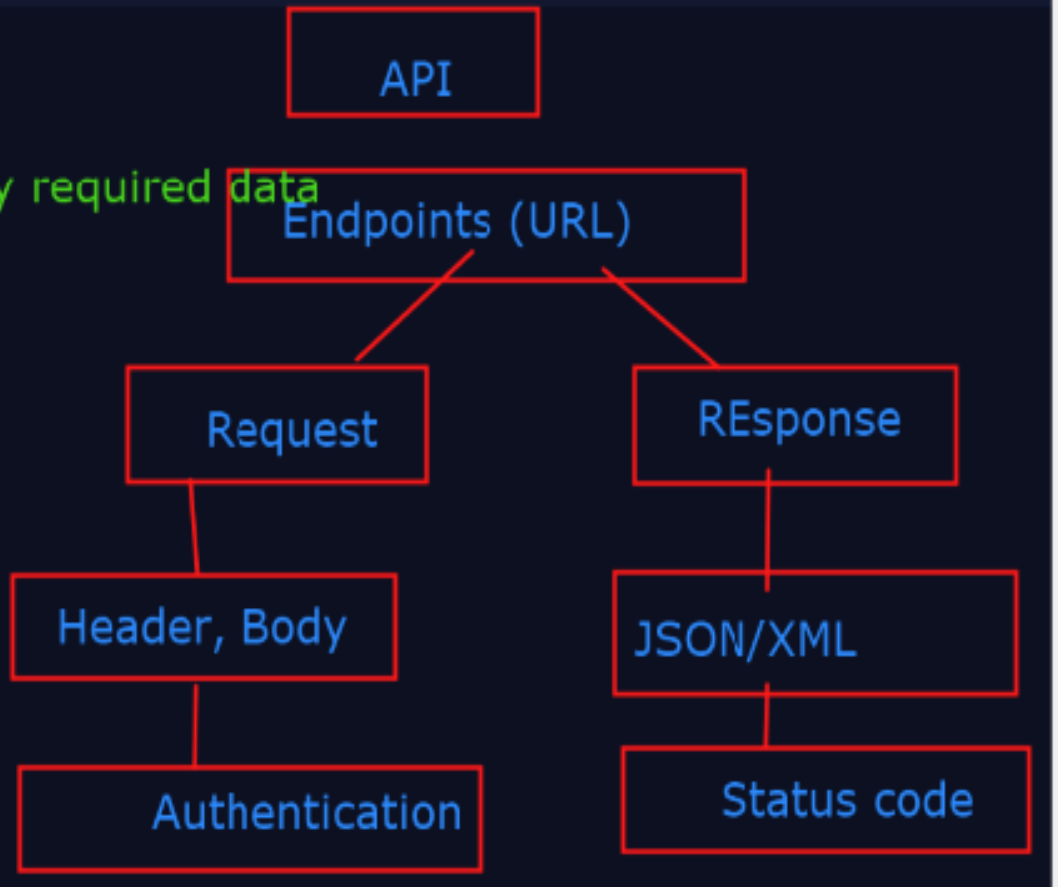
You are screen sharing
Stop share


Request
Response
Using Application
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Cloud-Based Services
Cras pretium iaculis nisi, consectetur volutpat metusda ut.
API
Server
Vestibulum vulputate augue sit amet risus rutrum tempus.
Database
Maecenas vel sem a lorem eleifend rhoncus id lacus.

communicate with each other.

1. Open API (Public API)
   -Accesible to everyone
2. Private API
   -Used within organization
   -used in restricted access
3. Partner API
   -Sharedonly with authorized person
4. Composite AP
   -combine multiple APIcalls into one requestI
5. REST API
   -Representational State Transfer

REST
SOAP: XML based
GraphQL:Client request only required data
gRPC:

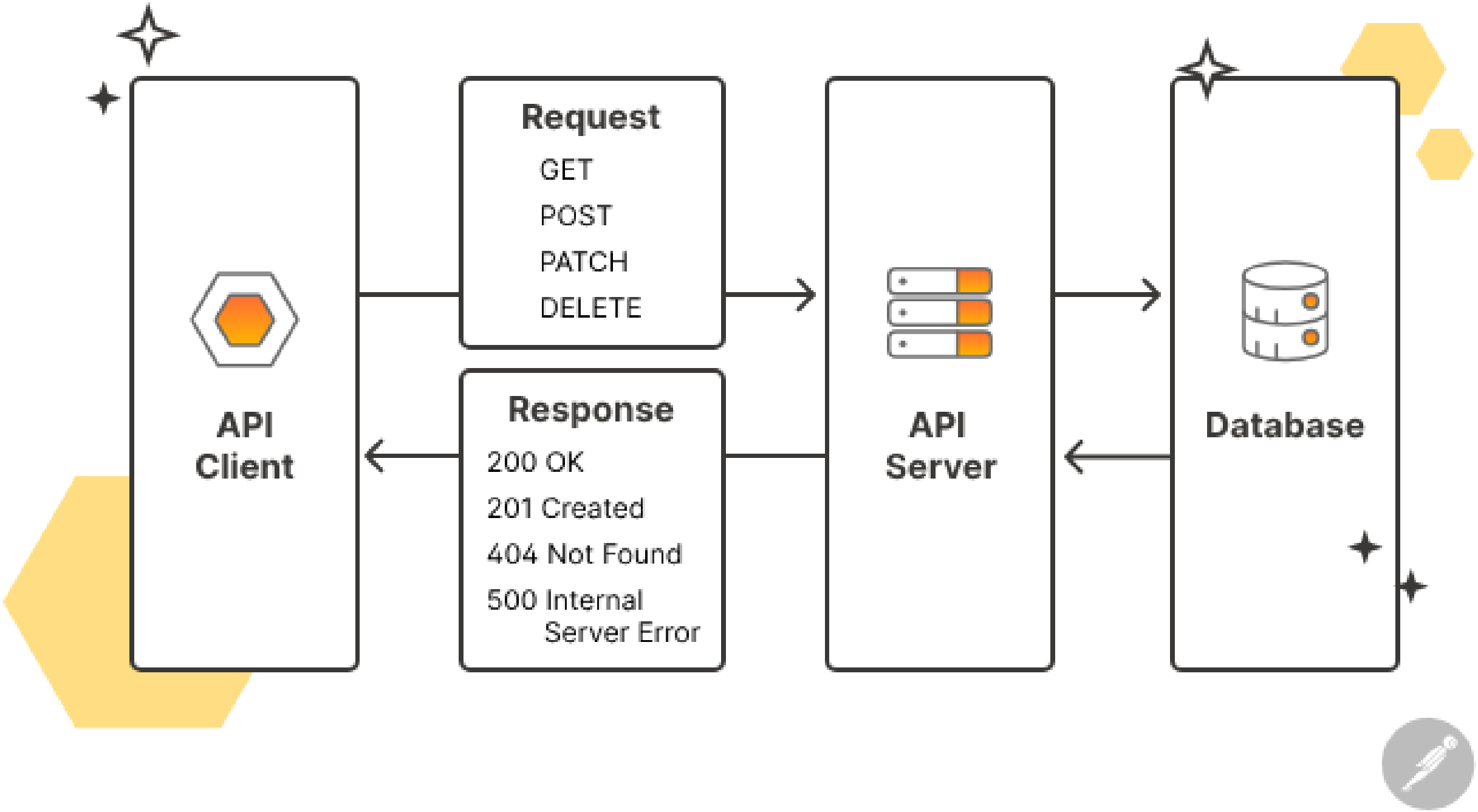


GET : Retrieve data
POST : Create new data
PUT :
DELETE :

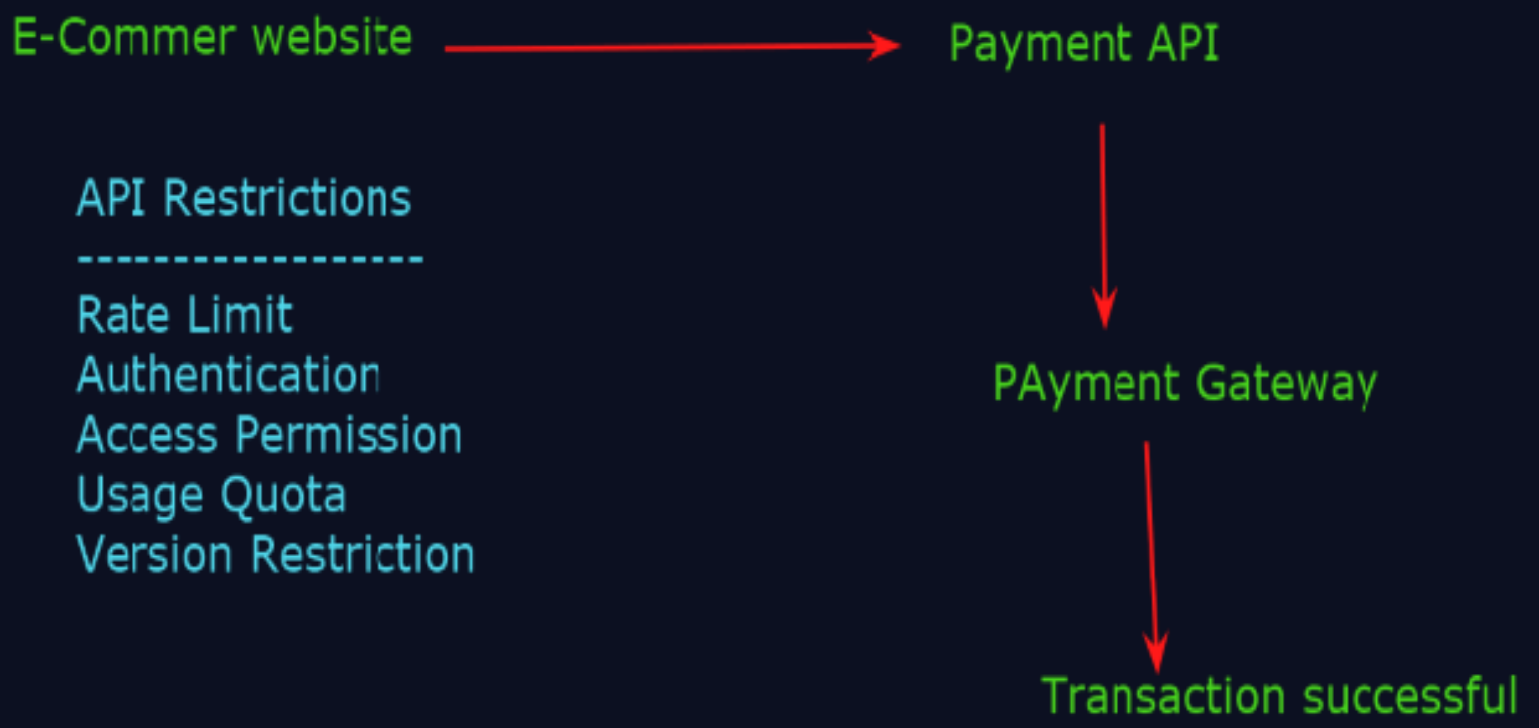
REquest: URL,HTTP method, HEader, Parameters, Body

Responce: JSON format

You are screen sharing
Stop share


Request
GET
POST
PATCH
DELETE
Response
200 OK
201 Created
404 Not Found
500 Internal Server Error
API Client
API Server
Database

communicate with each other.



You are screen sharing
Stop share
125

```
-------
Authorization: Bearer API_KEY
Content-Type: application/json

Body
-----
Model
Prompt
Temperature
Max Tokens

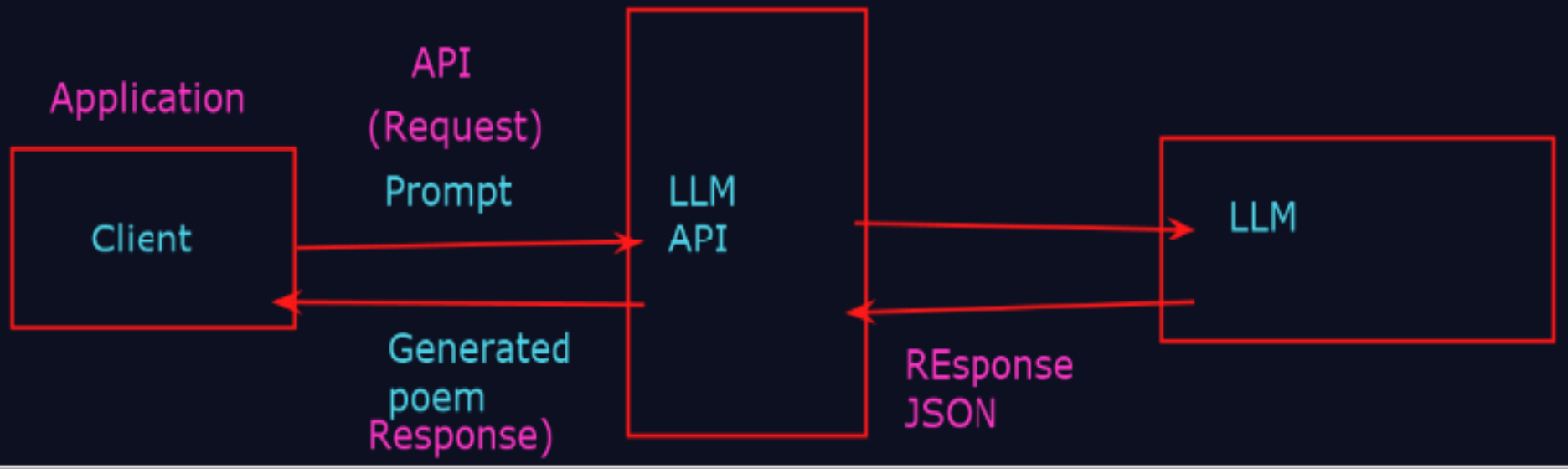
API REsponse:
---------------
Generated text
Token usage
Model information
Response status
```

API Providers

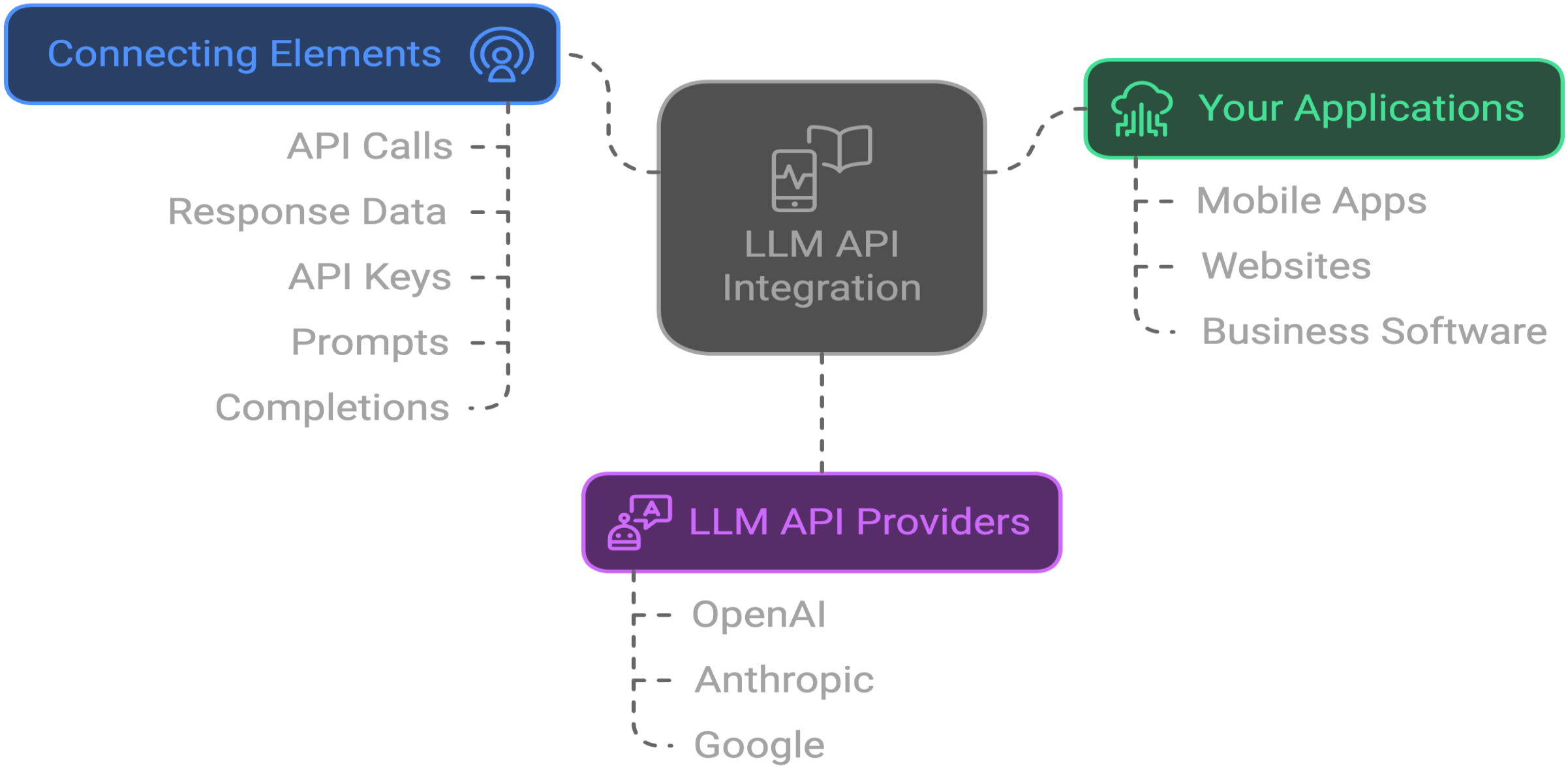
OpenAI ---> GPT series
Google ----> Gemini
Anthropic---->Claude
Meta------> Llama

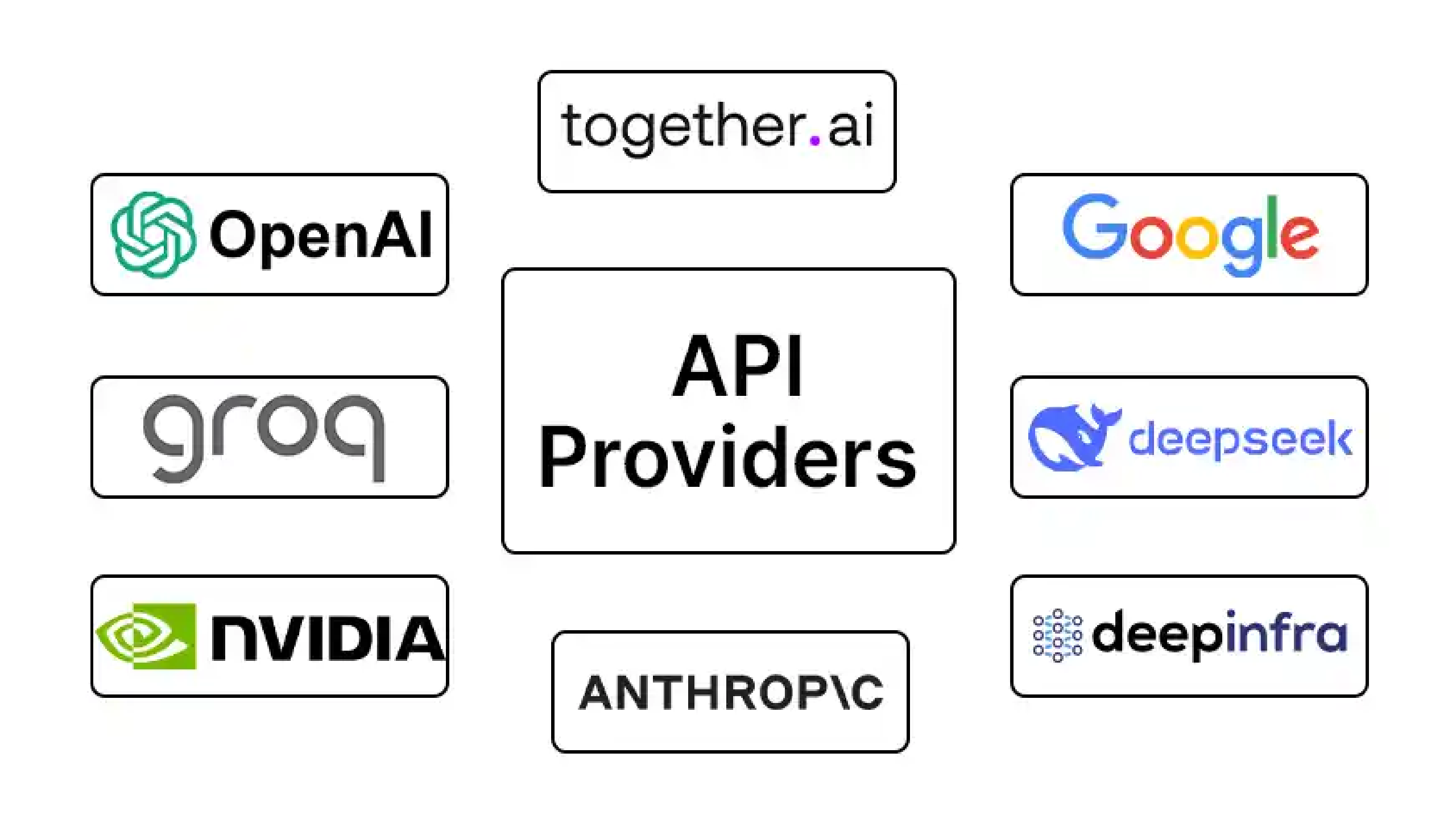
Componenets of LLM API

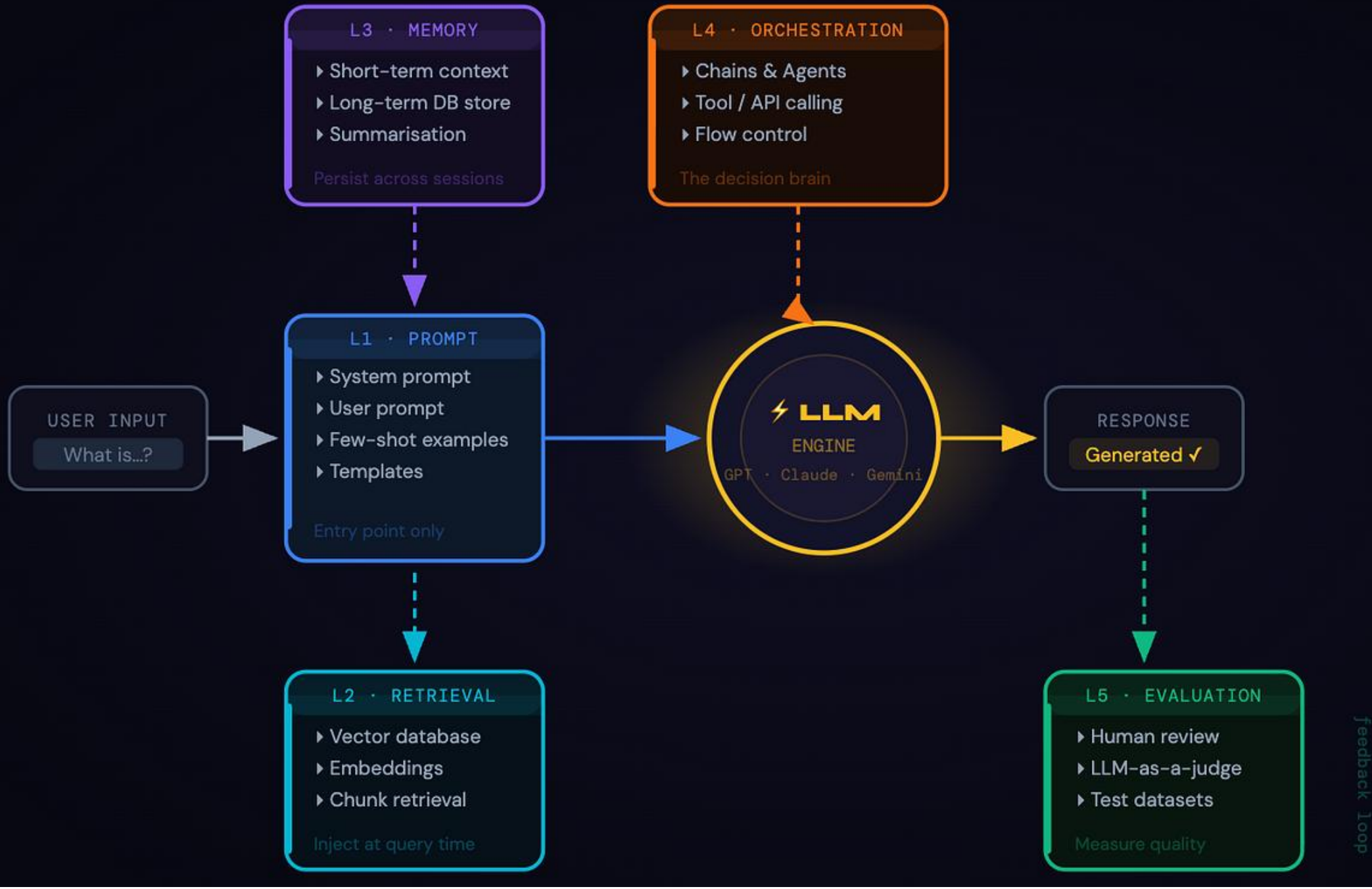
API Endpoints
API Key
Prompt
Model
Parameters
Response
Token usage



You are screen sharing
Stop share


Connecting Elements
API Calls
Response Data
API Keys
Prompts
Completions
LLM API Integration
Your Applications
Mobile Apps
Websites
Business Software
LLM API Providers
OpenAI
Anthropic
Google


together.ai
OpenAI
Google
API Providers
groq
deepseek
NVIDIA
deepinfra
ANTHROP\C


L3 · MEMORY
Short-term context
Long-term DB store
Summarisation
Persist across sessions
L4 · ORCHESTRATION
Chains & Agents
Tool / API calling
Flow control
The decision brain
USER INPUT
What is...?
L1 · PROMPT
System prompt
User prompt
Few-shot examples
Templates
Entry point only
LLM
ENGINE
GPT · Claude · Gemini
RESPONSE
Generated ✓
L2 · RETRIEVAL
Vector database
Embeddings
Chunk retrieval
Inject at query time
L5 · EVALUATION
Human review
LLM-as-a-judge
Test datasets
Measure quality
feedback loop

LLM APIs Key Features

Springs

01 Understanding of Context

02 Generation of Content

03 Customizability

04 Answering Questions

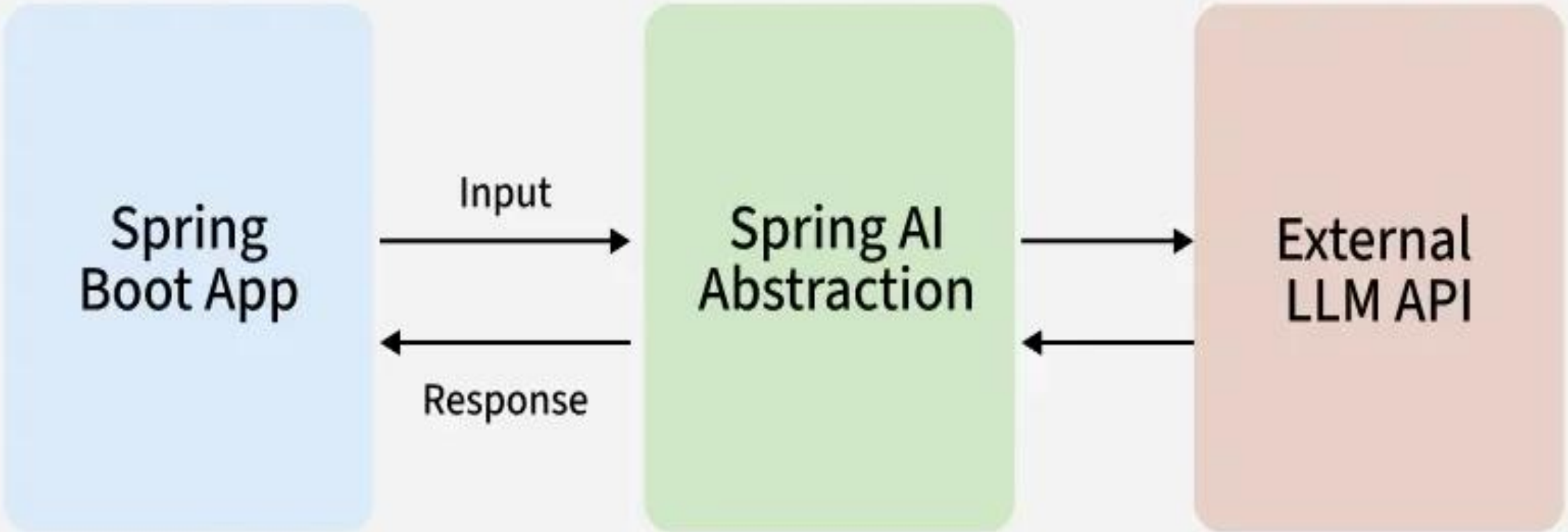
05 Translations

06 Sentiment Analysis

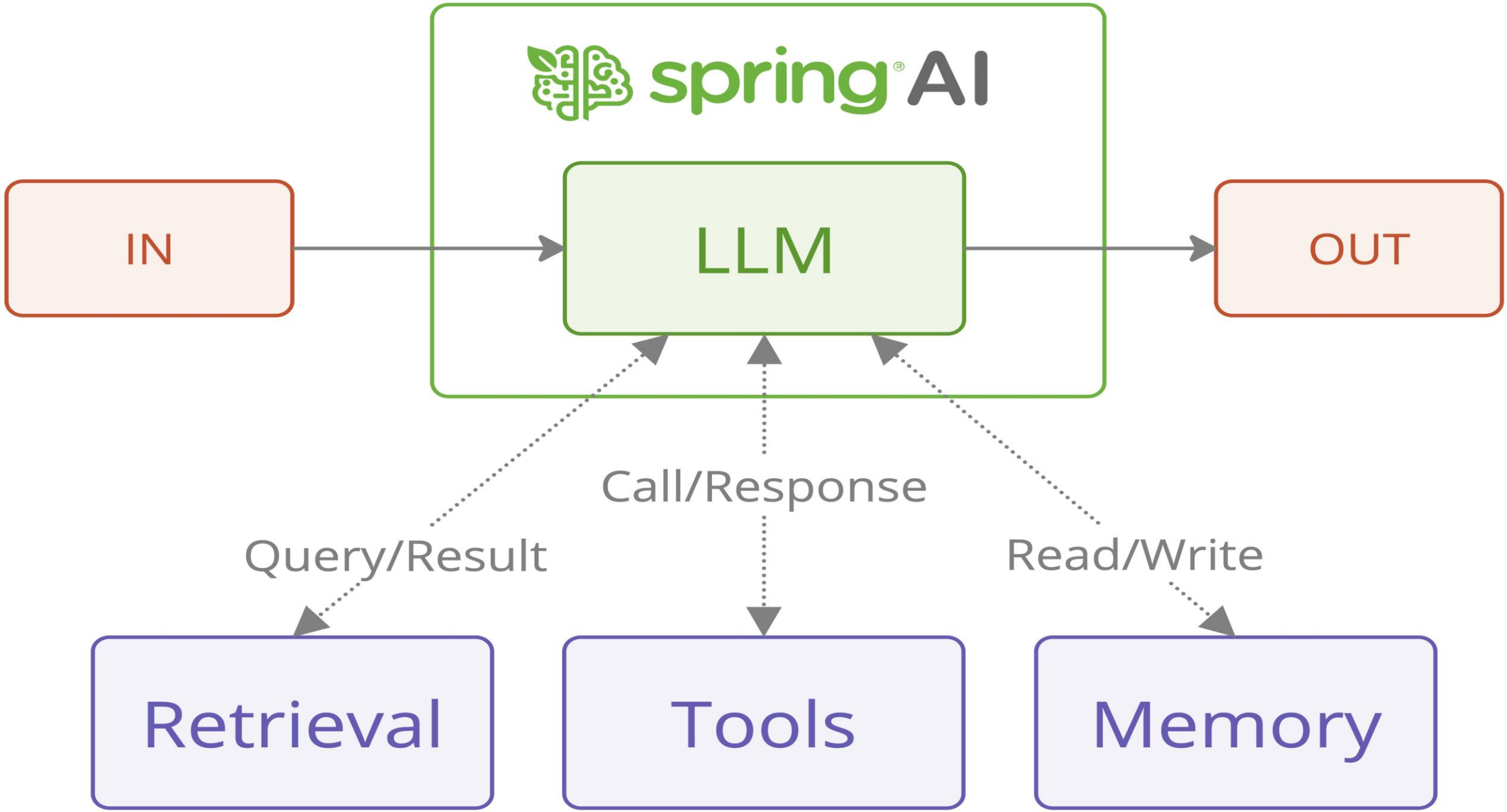
07 Multi-Lingual Support

08 Learning and Improvement

Spring AI Architecture



Application


spring AI
IN
LLM
OUT
Query/Result
Call/Response
Read/Write
Retrieval
Tools
Memory

Spring AI Integration with LLM

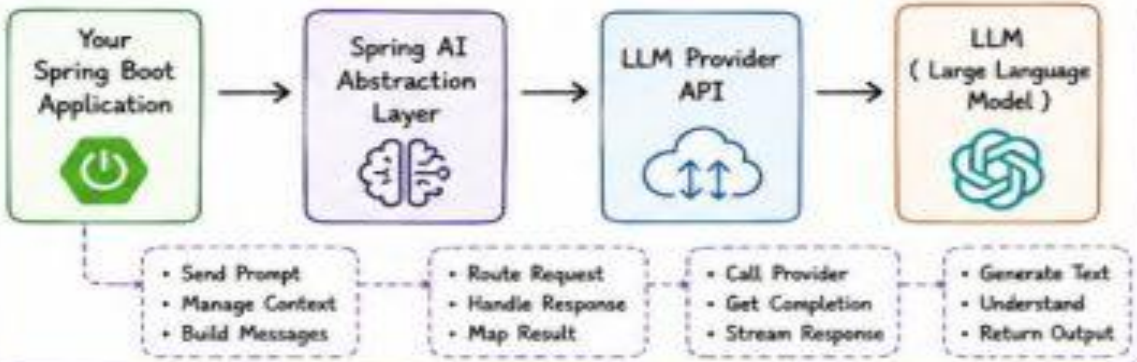
Connect, Configure and Chat with Large Language Models in your Spring Boot Applications



1. What is Spring AI?

- Spring AI simplifies integration with LLMs and AI services.
- It provides a unified abstraction layer for multiple AI providers.
- Focus on building AI-powered features, not the plumbing.
- Seamless integration with Spring Boot ecosystem.

2. How Spring AI Works (Architecture)



3. Supported LLM Providers

OpenAI (ChatGPT) · ANTHROPIC (Claude) · Google (Gemini) · Mistral AI

Ollama (Local LLM) · Azure OpenAI · ... and more

4. Quick Integration Steps

1. **Add Dependency** — Add Spring AI starter to your Maven project.
2. **Configure API Key** — Set API Key and model in application.yml
3. **Create AI Client** — Use ChatClient or EmbeddingClient provided by Spring AI.
4. **Send Prompt** — Send user prompt to LLM and get the response.
5. **Use Response** — Display or process the response in your application.

5. Example Code (Chat with LLM)

```
@Autowired
private ChatClient chatClient;

public String askAI(String question) {
    return chatClient.prompt()
        .user(question)
        .call()
        .content();
}
```

6. Example Configuration (application.yml)

```
spring:
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
      chat:
        options:
          model: gpt-4o
          temperature: 0.7
# For other providers, change the section
# (anthropic, google, mistral, ollama, etc.)
```

7. End-to-End Flow

1. **User** — User asks a question
2. **Prompt** — Question is sent to the application
3. **Spring AI Layer** — Spring AI processes and forwards the request
4. **LLM Provider** — LLM processes the prompt and generates a response
5. **Response** — Response is returned and used in the application

8. Common Use Cases

- AI Chatbots
- Code Assistant
- Document Q&A (RAG)
- Content Generation
- Summarization
- Data Extraction

9. Key Benefits

- Provider agnostic – switch LLMs easily.
- Minimal code changes.
- Unified API for chat, embeddings, and more.
- Built for Spring ecosystem.
- Easy to extend and customize.

Tip Start simple. Try a prompt, explore responses, and build intelligent applications with Spring AI!

Build Smarter. Build Faster. Build with Spring AI!

spring AI

FOUNDATIONS

Why a Proper Dev Environment Matters

Before writing prompts, get the surrounding tooling right — it's where most real-world friction happens.



Free Compute

Google Colab gives free GPU/TPU access — no local setup needed to experiment.



Shared Models

Hugging Face Hub hosts thousands of open models & datasets, ready to load.



Secure Keys

Proper key handling prevents leaked credentials and surprise bills.



Cost Awareness

Understanding tokens & pricing early avoids nasty API bill surprises.

Today's stack: Colab as your notebook, Hugging Face for open models, and OpenAI / Gemini / Groq for hosted API access.

Introduction to Hugging Face Hub

The largest open catalog of models, datasets, and demo "Spaces" — most loadable in a few lines of code.

load_model.py

```python
from transformers import pipeline

classifier = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-
           finetuned-sst-2-english"
)

result = classifier(
    "This course is incredibly useful!")

print(result)
# [{'label': 'POSITIVE', 'score': 0.999}]
```

Models

Pre-trained weights for text, vision, audio & more — filterable by task, size, and license.

Datasets

Thousands of ready-to-use datasets for fine-tuning or evaluation.

Spaces

Free-hosted demo apps (often Gradio/Streamlit) so you can try a model before writing any code.

Introduction to Google Colab

A free, browser-based Jupyter notebook with GPU/TPU access — today's workspace for every exercise.

Colab notebook cell

```
# Install SDKs for today's lab
!pip install -q openai google-genai groq

# Securely load a key from Colab Secrets
# (the key icon in the left sidebar)
from google.colab import userdata
api_key = userdata.get("OPENAI_API_KEY")

# Check which accelerator is attached
!nvidia-smi --query-gpu=name --format=csv
```

Free GPU/TPU

Runtime → Change runtime type gives free access to a T4 GPU for most exercises today.

Colab Secrets

Store API keys in the key-icon sidebar — never paste them directly into a cell.

Shareable & Reproducible

Notebooks run top-to-bottom and share easily via a link — ideal for labs and demos.

Understanding the LLM API Landscape

Three hosted APIs, each with a different edge — all reachable with a similar request/response shape.

OpenAI API

GPT-4o / GPT-4o-mini family

Broadest ecosystem & tooling; strong reasoning; widely supported by every framework.

Gemini API

Gemini 2.x family (Google)

Native multimodal (text, image, audio, video); generous free tier; huge context windows.

Groq API

Llama, Mixtral & others on LPU™

Purpose-built inference chips deliver extremely fast, low-latency token generation.

API Authentication & API Keys

Every request proves who's calling with a secret key sent as a Bearer token — treat it like a password.

HTTP request header

```
POST /v1/chat/completions HTTP/1.1
Host: api.openai.com
Authorization: Bearer sk-proj-**********
Content-Type: application/json

{"model": "gpt-4o-mini", "messages": [...]}
```

Every SDK builds this header for you — you just supply the key once at setup.

Do

- ✦ Store keys in environment variables or a secrets manager (Colab Secrets, .env + .gitignore)
- ✦ Scope keys to the minimum permissions needed; rotate them periodically
- ✦ Set usage limits/budgets in the provider dashboard as a safety net

Don't

- ✕ Never hardcode a key directly in a notebook cell or committed file
- ✕ Never share a key in a screenshot, chat message, or public repo

Official SDKs: Python

Each provider ships a Python client that wraps the raw HTTP calls into a few clean method calls.

call_llm.py

```python
from openai import OpenAI
from groq import Groq

# --- OpenAI ---
client = OpenAI(api_key=api_key)
resp = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Explain Top-P in 1 sentence."}],
)
print(resp.choices[0].message.content)

# --- Groq (same OpenAI-compatible shape) ---
groq_client = Groq(api_key=groq_key)
resp = groq_client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": "Explain Top-P in 1 sentence."}],
)
print(resp.choices[0].message.content)
```

Official SDKs: JavaScript / TypeScript

The same request shape, now from a Node.js or edge/serverless environment.

callLlm.ts

```ts
import { GoogleGenAI } from "@google/genai";

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

async function askGemini(question: string): Promise<string> {
  const response = await ai.models.generateContent({
    model: "gemini-2.5-flash",
    contents: question,
  });
  return response.text;
}

const answer = await askGemini("Explain Top-P in 1 sentence.");
console.log(answer);
```

PRODUCTION READINESS

Cost & Token Management

Every API call is billed by tokens in + tokens out — understanding this early prevents runaway costs.

usage from a response

```
print(resp.usage)

prompt_tokens:     412
completion_tokens: 128
total_tokens:      540
```

Roughly 4 characters ≈ 1 token in English — a 2,000-word doc is about 2,500–3,000 tokens.

Practical tips

- ✦ Set max_tokens to cap worst-case cost per call
- ✦ Cache repeated or near-identical requests
- ✦ Use a cheaper/faster model (e.g., mini or Groq) for simple tasks
- ✦ Log token usage per request to catch runaway prompts early

Illustrative pricing (per 1M tokens, input/output) — always check the provider's current page:

OpenAI (mini tier)	$0.15 / $0.60
Gemini (flash tier)	$0.075 / $0.30
Groq (open models)	$0.05–$0.79 / varies

DECISION GUIDE

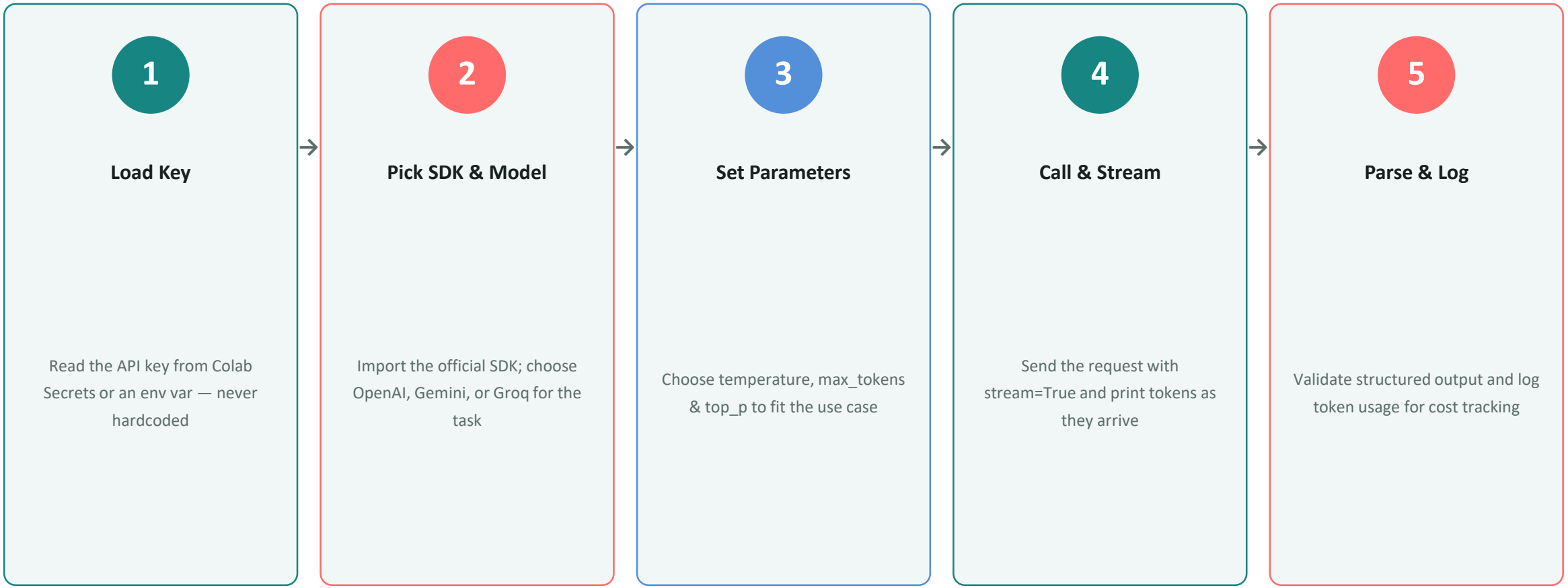
Choosing the Right API for the Job

Need	Recommended	Why	Watch Out For
Fastest inference	**Groq API**	Custom LPU hardware, very low latency for chat & agents	Smaller model catalog
Multimodal input	**Gemini API**	Native image/audio/video understanding, huge context	Newer ecosystem, fewer 3rd-party tools
Broadest tooling	**OpenAI API**	Most integrations, mature SDKs, wide framework support	Can be pricier at top-tier models
Full control / offline	**Hugging Face + local**	Open weights you can fine-tune, host, or run locally	You own the infra & scaling

: PRODUCTION READINESS

Hands-On: API Integration Lab

Bringing it all together — one small script that touches every concept from today.



KEY TAKEAWAYS

Your Environment Is Half of Building with GenAI

- ✓ Colab + Hugging Face give you a free, reproducible place to experiment with open models.
- ✓ OpenAI, Gemini, and Groq each excel at different things — pick based on the task, not habit.
- ✓ Streaming and structured outputs turn raw completions into production-ready responses.
- ✓ Temperature, Max Tokens, and Top-P — plus token awareness — keep quality and cost under control.

Thank You